



# NetFloor

Simulador de Cobertura Wi-Fi 2.5D e Diagnóstico de Campo Nativo

**Versão 8.1** revisão 8.1.0

Atualizada em 25/09/2026

---

**Stack:** Flutter / Dart · Kotlin (Android) · GitHub Pages (PWA) · JS Bridge

**Aplicativo:** [rogerdev5690.github.io/netfloor](https://rogerdev5690.github.io/netfloor)

**Código:** [github.com/Rogerdev5690/netfloor](https://github.com/Rogerdev5690/netfloor) · [github.com/Rogerdev5690/netfloor-shell](https://github.com/Rogerdev5690/netfloor-shell)

# Sumário

---

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>1. Visão Geral e Proposta do Produto</b>             | <b>4</b>  |
| 1.1 Conceito                                            | 4         |
| 1.2 Casos de uso                                        | 4         |
| 1.3 Módulos em resumo                                   | 4         |
| 1.4 Arquitetura Serverless / PWA                        | 5         |
| <b>2. Arquitetura de Software e Stack Tecnológica</b>   | <b>6</b>  |
| 2.1 Stack                                               | 6         |
| 2.2 Repositórios e estrutura de pastas                  | 6         |
| 2.3 Frontend Web/PWA (lib/main.dart)                    | 7         |
| 2.4 Android Shell                                       | 8         |
| 2.5 Ponte de Comunicação (JS Bridge)                    | 8         |
| <b>3. Motor do Simulador e Engenharia de RF</b>         | <b>10</b> |
| 3.1 Fórmula de propagação                               | 10        |
| 3.2 Catálogo de hardware                                | 10        |
| 3.3 Distância 3D e escala                               | 10        |
| 3.4 Física de barreiras (ray-casting)                   | 11        |
| 3.5 Arquitetura 2.5D multi-pavimento                    | 11        |
| 3.6 Exemplos numéricos                                  | 12        |
| 3.7 Visualização do mapa de calor                       | 12        |
| 3.8 Interação                                           | 13        |
| 3.9 Limitações do modelo                                | 13        |
| 3.10 Simulação em 2.4 / 5 / 6 GHz                       | 14        |
| <b>4. Módulo de Diagnóstico Mobile (Android Nativo)</b> | <b>15</b> |
| 4.1 Modos de operação                                   | 15        |
| 4.2 Requisitos e permissões do Android                  | 15        |
| 4.3 Aba Espectro e Saúde do Canal                       | 15        |
| 4.4 Aba Sinal (walk-through)                            | 16        |
| 4.5 Aba Latência e Rede (ping duplo)                    | 16        |
| 4.6 Controlador de diagnóstico                          | 17        |
| 4.7 Modo simulação                                      | 17        |
| 4.8 Estado de validação                                 | 17        |
| <b>5. NetFloor Enterprise (v8.0)</b>                    | <b>18</b> |
| 5.1 Visão geral                                         | 18        |
| 5.2 Fluxo de trabalho típico em campo                   | 18        |
| 5.3 Ferramentas do mapa e gestos                        | 18        |
| 5.4 Modos de visualização                               | 19        |
| 5.5 Pontos de medição e medição de campo                | 19        |
| 5.6 Persistência offline-first e arquivo .json          | 19        |
| 5.7 Laudo de vistoria em PDF                            | 20        |
| 5.8 Assinatura digital do cliente                       | 21        |
| 5.9 Entrega de arquivos (PDF e .json)                   | 21        |
| 5.10 Estado de validação dos recursos da v8.0           | 21        |
| <b>6. Gestão de Plantas e Mídia</b>                     | <b>23</b> |
| 6.1 Modelo de dados                                     | 23        |
| 6.2 Cadeia de resolução da imagem                       | 23        |
| 6.3 Biblioteca atual                                    | 23        |
| 6.4 Como adicionar uma nova planta                      | 24        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 6.5 Upload em memória (UInt8List) .....                           | 24        |
| 6.6 Política de conteúdo e licenças .....                         | 24        |
| 6.7 Histórico de alterações .....                                 | 25        |
| <b>7. Guia de Instalação, Build e CI/CD .....</b>                 | <b>27</b> |
| 7.1 Ambiente de desenvolvimento (Windows 11) .....                | 27        |
| 7.2 Dependências (pubspec.yaml) .....                             | 27        |
| 7.3 Permissões do Android (AndroidManifest.xml do Shell) .....    | 27        |
| 7.4 Desenvolvimento local .....                                   | 28        |
| 7.5 Build web de produção .....                                   | 28        |
| 7.6 Pipeline de implantação contínua (OTA) via GitHub Pages ..... | 28        |
| 7.7 Build e versionamento do APK Shell .....                      | 29        |
| 7.8 Checklist de release .....                                    | 29        |
| 7.9 Solução de problemas .....                                    | 30        |
| 7.10 Segurança e privacidade .....                                | 30        |
| <b>8. Limitações Conhecidas e Pendências .....</b>                | <b>31</b> |
| <b>9. Manutenção desta Documentação (Regra de Projeto) .....</b>  | <b>32</b> |
| <b>Apêndice A — Glossário .....</b>                               | <b>33</b> |
| <b>Apêndice B — Referência rápida de constantes .....</b>         | <b>34</b> |

# 1. Visão Geral e Proposta do Produto

## 1.1 Conceito

O **NetFloor** é um aplicativo com duas frentes complementares:

1. **Simulador de Cobertura Wi-Fi.** O usuário escolhe uma planta baixa, posiciona roteadores de modelos reais (com potência de transmissão própria) e vê, em tempo real, um **mapa de calor** de sinal sobreposto à planta. O cálculo considera a distância em 3D, a atenuação por paredes (ray-casting) e a atenuação por lajes entre pavimentos.
2. **Diagnóstico de Campo Nativo (NetFloor Diagnostic).** Uma aba que, quando executada dentro do aplicativo Android (NetFloor Shell), lê os dados **reais** do aparelho: espectro de canais em 2.4 GHz e 5 GHz com saúde do canal, sinal (RSSI) em tempo real e latência (ping duplo: roteador local versus DNS da Internet), além da velocidade PHY negociada. Fora do Shell (navegador comum), a aba funciona em **modo simulação** com dados fictícios, claramente identificados.

A **versão 8.0 (Enterprise)** acrescenta recursos de campo e de relacionamento com o cliente: **laudo de vistoria em PDF** com assinatura digital, **calibração de escala por régua**, **materiais de parede** com perdas realistas, **simulação em 2.4 / 5 / 6 GHz**, **zoom e deslocamento** do mapa, **modos de visualização** (Apresentação e Diagnóstico) e **armazenamento offline** dos projetos com exportação/importação em `.json` (capítulo 5).

O nome da marca exibida ao usuário é **NetFloor**.

## 1.2 Casos de uso

| Perfil                        | Uso típico                                                                                                                         |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Instalador / técnico de redes | Planejar onde posicionar roteadores ou APs antes da instalação; comparar modelos (Huawei, TP-Link Deco, UniFi).                    |
| Morador / pequeno escritório  | Entender por que o Wi-Fi cai em certos cômodos; testar cobertura em sobrados e prédios com vários andares.                         |
| Diagnóstico em campo          | Caminhar pelo ambiente observando o RSSI (walk-through), escolher o melhor canal e medir latência até o roteador e até a Internet. |

## 1.3 Módulos em resumo

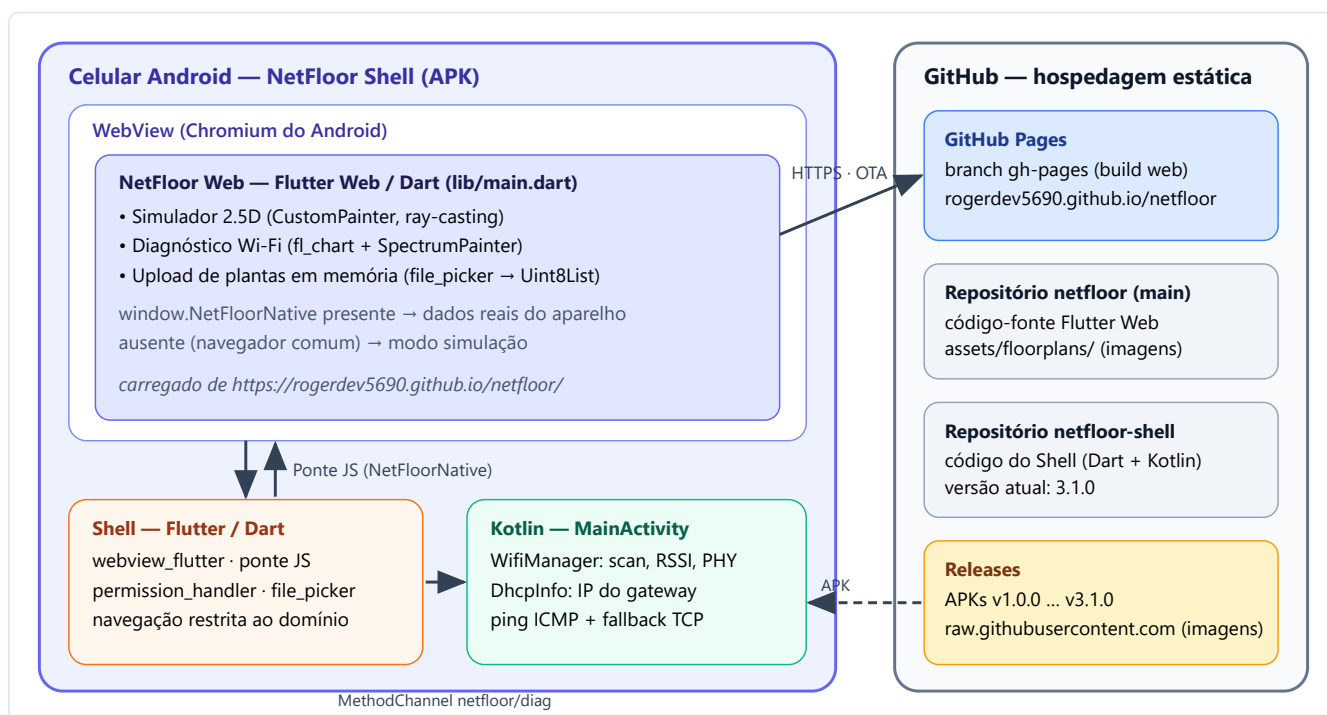
| Módulo                                                     | Onde roda                                                    | Fonte dos dados                                  |
|------------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------|
| Simulador 2.5D (mapa de calor)                             | Navegador, PWA e Shell Android                               | Modelo matemático (seção 3)                      |
| Espectro de canais e saúde do canal                        | Shell Android (real); navegador (simulação)                  | <code>WifiManager.getScanResults()</code>        |
| Monitor de sinal (walk-through)                            | Shell Android (real); navegador (simulação)                  | <code>WifiInfo.getRssi()</code> a cada 1 s       |
| Latência e rede (ping duplo)                               | Shell Android (real); navegador (simulação)                  | ping do sistema; <code>DhcpInfo</code>           |
| Biblioteca e upload de plantas                             | Todos os ambientes                                           | Assets, rede (GitHub) e bytes em memória         |
| Ferramentas de mapa (paredes, régua, pontos), banda e zoom | Todos os ambientes                                           | Toque/mouse do usuário (seção 5.3)               |
| Projetos offline e arquivo <code>.json</code>              | Todos os ambientes                                           | IndexedDB do navegador/WebView (seção 5.6)       |
| Laudo de vistoria em PDF com assinatura                    | Todos os ambientes; salvar/compartilhar nativo no Shell 3.2+ | Simulador + medições de campo (seções 5.7 a 5.9) |

## 1.4 Arquitetura Serverless / PWA

O NetFloor **não possui servidor próprio, banco de dados ou API**. Todo o produto é composto de arquivos estáticos:

- O **aplicativo web** (Flutter Web compilado para JavaScript) é publicado em **GitHub Pages**, no endereço <https://rogerdev5690.github.io/netfloor/>.
- O **NetFloor Shell** é um APK Android que abre esse endereço dentro de uma **WebView** e acrescenta capacidades nativas (Wi-Fi, ping, seletor de arquivos) por meio de uma ponte JavaScript.
- Toda a lógica de negócio vive no navegador/WebView; nada é enviado a servidores de terceiros além do carregamento de arquivos estáticos do GitHub. Projetos, laudos e assinaturas ficam **no próprio aparelho** (IndexedDB) e só saem dele quando o usuário exporta ou compartilha um arquivo.

**Atualização OTA (over-the-air).** Como a interface vem do site, publicar uma nova versão do site atualiza o aplicativo instalado no celular na próxima abertura, sem reinstalar o APK. O APK só precisa ser reinstalado quando algo **nativo** muda (código Kotlin, permissões, versão do Shell). Para garantir isso, o build web usa `--pwa-strategy=none` (sem cache agressivo de service worker).



Arquitetura geral do NetFloor

## 2. Arquitetura de Software e Stack Tecnológica

### 2.1 Stack

| Camada               | Tecnologia                                                                    | Versão / observação                  | Função                                                                                                       |
|----------------------|-------------------------------------------------------------------------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Frontend Web/PWA     | <b>Flutter / Dart</b>                                                         | Flutter 3.47.4 (stable), Dart 3.13.3 | Interface, simulador, diagnóstico                                                                            |
| Renderização 2D      | <code>CustomPainter</code> , <code>Canvas</code> , <code>ImageFiltered</code> | SDK Flutter                          | Mapa de calor, ícones dos roteadores, espectro                                                               |
| Gráficos de linha    | <code>fl_chart</code>                                                         | ^1.2.0                               | Sinal (RSSI) e latência                                                                                      |
| Upload de arquivos   | <code>file_picker</code>                                                      | ^13.1.0                              | Plantas, logo e projetos <code>.json</code> em memória ( <code>Uint8List</code> )                            |
| Laudo em PDF         | <code>pdf</code> (100 % Dart)                                                 | ^3.13.1                              | Geração do laudo no navegador/WebView                                                                        |
| Imagens              | <code>image</code>                                                            | ^4.10.1                              | Codificação JPEG dos mapas de calor do laudo                                                                 |
| Armazenamento local  | <code>sembast</code> + <code>sembast_web</code>                               | ^3.8.11 / ^2.4.6                     | Banco NoSQL sobre IndexedDB (offline-first)                                                                  |
| Download de arquivos | <code>web</code> ( <code>package:web</code> )                                 | ^1.1.1                               | <code>Blob</code> + âncora <code>download</code> no navegador                                                |
| Fonte do PDF         | Roboto (Apache 2.0)                                                           | <code>assets/fonts/</code>           | Acentos e símbolos no laudo                                                                                  |
| Interop Web          | <code>dart:js_interop</code> , <code>dart:js_interop_unsafe</code>            | SDK Dart                             | Ponte com o Shell                                                                                            |
| Android Shell (UI)   | <b>Flutter / Dart</b>                                                         | Shell 3.2.0 (build 5)                | Hospeda a WebView e a ponte                                                                                  |
| WebView              | <code>webview_flutter</code> + <code>webview_flutter_android</code>           | ^4.10.0 / ^4.14.1                    | Exibe o site e injeta <code>NetFloorNative</code>                                                            |
| Permissões           | <code>permission_handler</code>                                               | 12.0.1                               | Localização e Nearby Wi-Fi                                                                                   |
| Android nativo       | <b>Kotlin</b>                                                                 | JDK 17 (Temurin 17.0.20)             | <code>WifiManager</code> , ping, <code>DhcpInfo</code> , <code>MediaStore</code> , <code>FileProvider</code> |
| Build Android        | Gradle + AGP (template Flutter)                                               | SDK 36, build-tools 35/36            | Empacotamento do APK                                                                                         |
| Hospedagem           | <b>GitHub Pages</b>                                                           | branch <code>gh-pages</code>         | Servir o app web                                                                                             |
| Distribuição do APK  | <b>GitHub Releases</b>                                                        | <code>gh release create</code>       | Baixar/installar o Shell                                                                                     |
| Automação local      | PowerShell, Bash, <code>gh CLI</code> , <code>git</code>                      | Windows 11                           | Build, deploy e versionamento                                                                                |

### 2.2 Repositórios e estrutura de pastas

| Repositório                                                                   | Conteúdo                                                                               |
|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>github.com/Rogerdev5690/netfloor</code> (branch <code>main</code> )     | Código-fonte do app web, assets das plantas e esta documentação ( <code>docs/</code> ) |
| <code>github.com/Rogerdev5690/netfloor</code> (branch <code>gh-pages</code> ) | Build web publicado (gerado, sobrescrito a cada deploy)                                |

| Repositório                                                                                                                     | Conteúdo                                |
|---------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| <a href="https://github.com/Rogerdev5690/netfloor-shell">github.com/Rogerdev5690/netfloor-shell</a> (branch <code>main</code> ) | Código do Shell Android (Dart + Kotlin) |
| <a href="https://github.com/Rogerdev5690/netfloor/releases">github.com/Rogerdev5690/netfloor/releases</a>                       | APKs do Shell (v1.0.0 a v3.2.0)         |

Estrutura local ( `C:\Users\Roger\Desktop\Nova pasta\` ):

```

netfloor/                # app web (Flutter)
  lib/main.dart          # todo o código do app (arquivo único, ~6.730 linhas)
  assets/floorplans/     # imagens das plantas
  assets/fonts/         # Roboto (Regular, Bold, Italic) usada no PDF
  pubspec.yaml
  run_web.bat           # servidor de desenvolvimento (porta 8090)
  docs/                 # esta documentação
    DOCUMENTACAO.md     # fonte da documentação
    build_pdf.py        # gerador do PDF
    diagramas/*.svg     # diagramas
    Documentacao_Tecnica_App.pdf
  android/              # legado: build nativo da v1.0.0 (não usado)
netfloor_shell/         # Shell Android
  lib/main.dart         # WebView + ponte JS
  android/app/src/main/
    AndroidManifest.xml # permissões
    kotlin/com/netfloor/netfloor_shell/MainActivity.kt

```

## 2.3 Frontend Web/PWA ( `lib/main.dart` )

O arquivo único é dividido em três partes:

| Parte                 | Conteúdo                                                                              | Classes principais                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Raiz                  | Aplicativo, tema, estado compartilhado e navegação inferior (Simulador / Diagnóstico) | <code>NetFloorApp</code> , <code>NetFloorShell</code>                                                                                                                                                                                                                                                                                                                                                                    |
| Parte 1 — Simulador   | Catálogo, plantas, materiais, bandas, física de sinal, pintura, ferramentas e tela    | <code>RouterModelSpec</code> , <code>FloorPlanDef</code> , <code>FloorState</code> , <code>WallSegment</code> , <code>RfBandSpec</code> , <code>ProjectDef</code> , <code>NetworkModel</code> , <code>SignalField</code> , <code>HeatmapPainter</code> , <code>ToolOverlayPainter</code> , <code>FloorPlanPainter</code> , <code>RouterDevicePainter</code> , <code>RadarPingPainter</code> , <code>SimulatorPage</code> |
| Parte 2 — Diagnóstico | Ponte nativa, modelos de dados, fontes de dados, controlador e três abas              | <code>NativeBridge</code> , <code>WifiNetwork</code> , <code>LinkInfo</code> , <code>DiagnosticsSource</code> , <code>NativeBridgeDiagnostics</code> , <code>SimulatedDiagnostics</code> , <code>DiagnosticsController</code> , <code>DiagnosticPage</code> , <code>SpectrumTab</code> , <code>SpectrumPainter</code> , <code>SignalTab</code> , <code>LatencyTab</code>                                                 |
| Parte 3 — Enterprise  | Temas, armazenamento local, entrega de arquivos, assinatura, tela e gerador do laudo  | <code>ViewMode</code> , <code>buildTheme</code> , <code>AppStore</code> , <code>FileIO</code> , <code>SignaturePadPage</code> , <code>ReportPage</code> , <code>LaudoPdf</code> , <code>renderFloorRaster</code>                                                                                                                                                                                                         |

**Gerenciamento de estado.** Usa-se `ChangeNotifier` com `AnimatedBuilder`, sem bibliotecas externas. `NetworkModel` guarda projeto, pavimentos (com escala e paredes), roteadores, pontos de medição, registros de diagnóstico e dados do laudo; `DiagnosticsController` guarda varredura, amostras de sinal e de ping. O `NetFloorShell` é dono dos dois e do `AppStore` (armazenamento), repassando-os às telas. Durante o arraste de um roteador só a camada do mapa de calor é redesenhada, mantendo a interação fluida.

**Composição visual.** A área do mapa fica dentro de um `InteractiveViewer` (zoom e deslocamento) e é um `Stack` de camadas: (1) imagem ou desenho da planta (com filtro de cor conforme o modo de visualização), (2) mapa de calor com

desfoque, (3) paredes desenhadas e régua, (4) anéis pulsantes dos roteadores, (5) roteadores de outros andares (esmaecidos), pontos de medição e roteadores do andar, com tamanho constante na tela. Por cima, fora do zoom, ficam o selo de escala, a legenda e os botões de zoom.

**Navegação.** Uma `NavigationBar` alterna entre Simulador e Diagnóstico dentro de um `IndexedStack`, preservando o estado das duas telas. Cada tela tem seu `TickerMode`, de modo que animações e temporizadores da aba oculta ficam parados.

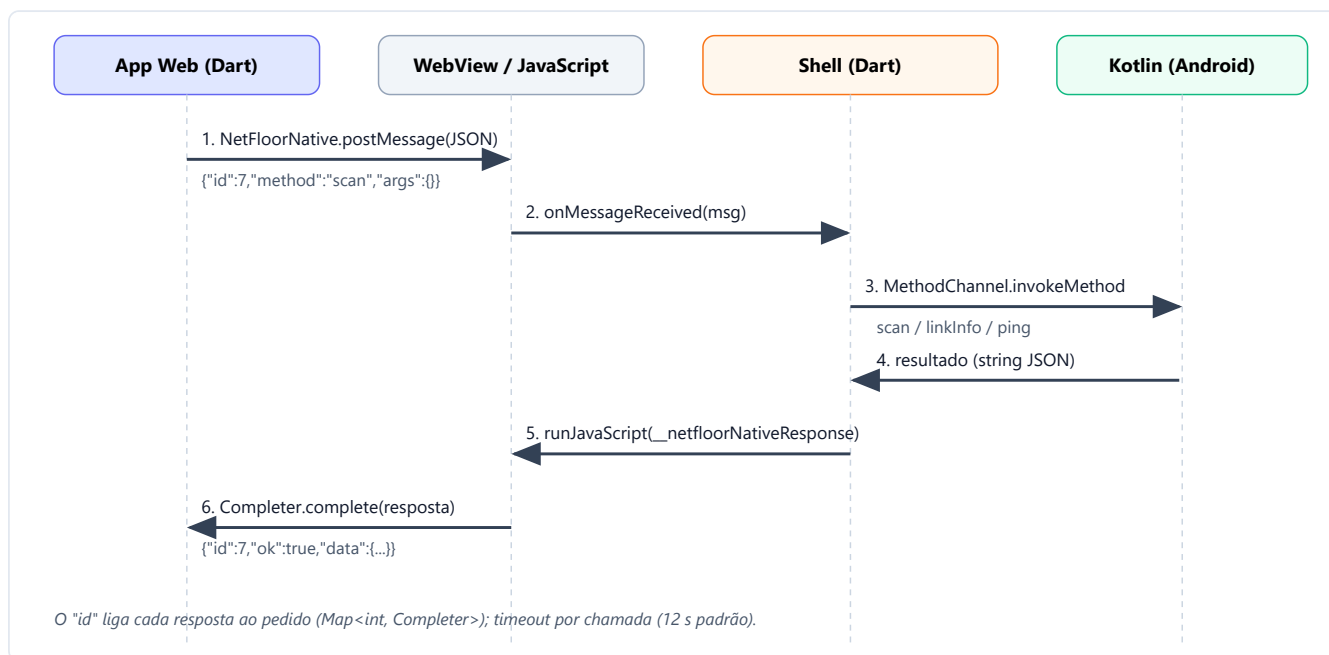
## 2.4 Android Shell

O Shell é um aplicativo Flutter mínimo cujo corpo é uma `WebView` em tela cheia.

| Responsabilidade                | Implementação                                                                                                                                                                                                                                             |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Carregar o app                  | <code>WebViewController.loadRequest</code> para <a href="https://rogerdev5690.github.io/netfloor/">https://rogerdev5690.github.io/netfloor/</a>                                                                                                           |
| Proteger a ponte                | <code>onNavigationRequest</code> só permite o host <a href="https://rogerdev5690.github.io">rogerdev5690.github.io</a> ; qualquer outra navegação é bloqueada                                                                                             |
| Tratar falha de rede            | Tela "Não foi possível carregar" com botão de nova tentativa (apenas para erros do quadro principal)                                                                                                                                                      |
| Expor a ponte                   | <code>addJavaScriptChannel('NetFloorNative', ...)</code>                                                                                                                                                                                                  |
| Permissões em tempo de execução | <code>permission_handler</code> : <code>locationWhenInUse</code> e <code>nearbyWifiDevices</code>                                                                                                                                                         |
| Seletor de arquivos             | <code>AndroidWebViewController.setOnShowFileSelector</code> chama <code>FilePicker.pickFile / pickFiles</code> e devolve URIs; quando a página pede <code>.json</code> (importação de projeto), abre o seletor filtrado por JSON                          |
| Entrega de arquivos (3.2.0)     | Método <code>saveFile</code> da ponte: remonta o arquivo recebido em pedaços de base64 e o entrega ao Kotlin, que grava em <b>Downloads/NetFloor</b> ( <code>MediaStore</code> ) ou abre a <b>folha de compartilhamento</b> ( <code>FileProvider</code> ) |
| Acesso ao Wi-Fi                 | <code>MethodChannel('netfloor/diag')</code> para o Kotlin                                                                                                                                                                                                 |

## 2.5 Ponte de Comunicação (JS Bridge)

A ponte é **bidirecional** e assíncrona, baseada em mensagens JSON com identificador de correlação.



Sequência da ponte Kotlin ↔ Flutter Web

## Formato das mensagens

```
// App Web -> Shell (NetFloorNative.postMessage)
{ "id": 7, "method": "ping", "args": { "host": "8.8.8.8" } }

// Shell -> App Web (window.__netfloorNativeResponse)
{ "id": 7, "ok": true, "data": { "ms": 21.4 } }
{ "id": 7, "ok": false, "error": "mensagem de erro" }
```

## Métodos disponíveis

| Método      | Argumentos                                      | Resposta ( <b>data</b> )                                            | Timeout no app web    |
|-------------|-------------------------------------------------|---------------------------------------------------------------------|-----------------------|
| hello       | —                                               | { platform, shellVersion }                                          | 12 s                  |
| permissions | —                                               | { location, nearby, locationServices }<br>(booleanos)               | 90 s (usuário decide) |
| scan        | —                                               | { networks: [...], scanStarted, error? }                            | 20 s                  |
| linkInfo    | —                                               | { connected, ssid, bssid, rssi, linkSpeed, frequency, gateway, ip } | 12 s                  |
| ping        | host                                            | { ms } (null se perdido)                                            | 8 s                   |
| saveFile    | name, mime, share, chunk, chunks, data (base64) | { done } e, no último pedaço, { done: true, location }              | 60 s por pedaço       |

### Como funciona.

1. No lado web, `NativeBridge.available` é verdadeiro quando existe `window.NetFloorNative` (injetado pela WebView do Shell). Cada chamada gera um `id` incremental e guarda um `Completer` em um mapa.
2. A resposta chega pela função global `window.__netfloorNativeResponse`, registrada pelo próprio app web na primeira chamada. O `id` localiza o `Completer` correspondente.
3. No Shell, a resposta é serializada duas vezes (`jsonEncode(jsonEncode(...))`) para formar um literal de string JavaScript válido antes de `runJavaScript`.
4. Estourando o tempo limite, a chamada lança `TimeoutException` e a UI mostra o erro da respectiva aba.

**Envio de arquivos (`saveFile`).** Mensagens grandes na ponte JS são frágeis; por isso o app web divide o arquivo em pedaços de 256 KB de base64 (`chunk 0` a `chunks - 1`). O Shell acumula os pedaços e, no último, decodifica os bytes e os passa por `MethodChannel` ao Kotlin. Com `share = false` o arquivo vai para `Downloads/NetFloor` (Android 10+: `MediaStore.Downloads`; antes: pasta de downloads do app, sem exigir permissão de armazenamento); com `share = true` é copiado para o cache e aberto na folha de compartilhamento (WhatsApp, e-mail, Drive...). O app web só usa esse caminho se `hello` informar `Shell ≥ 3.2.0`; em navegador comum faz o download normal por `Blob`.

**Segurança.** A ponte só existe na WebView do Shell, e o Shell bloqueia navegação para domínios diferentes do GitHub Pages do projeto. Métodos desconhecidos são rejeitados (`UnsupportedError`). O `ping` valida o nome do host por expressão regular antes de executar.

## 3. Motor do Simulador e Engenharia de RF

### 3.1 Fórmula de propagação

Para cada ponto  $(x, y)$  do pavimento exibido, o sinal é o **máximo** entre os roteadores da rede (união da cobertura de uma rede mesh):

$$S(x, y) = \max_i \left( P_{tx,i} + \Delta_{ref}(b) - k(b) \cdot \log_{10}(d_{3D,i}) - f_{par}(b) \cdot \sum L_{parede} - L_{laje,i} \right)$$

Onde:

| Símbolo           | Significado                                                                          |
|-------------------|--------------------------------------------------------------------------------------|
| $P_{tx,i}$        | Potência de transmissão do roteador $i$ , em dBm, definida pelo modelo (seção 3.2)   |
| $b$               | Banda simulada: 2,4, 5 ou 6 GHz (seção 3.10)                                         |
| $\Delta_{ref}(b)$ | Ajuste do nível a 1 m em relação a 2,4 GHz (0, -6,4 e -7,9 dB)                       |
| $k(b)$            | Coefficiente de perda de percurso: $10 \cdot n$ (22, 24 e 25)                        |
| $d_{3D,i}$        | Distância tridimensional entre o roteador e o ponto, em metros reais (mínimo de 1 m) |
| $f_{par}(b)$      | Fator de opacidade das paredes na banda (1,00, 1,35 e 1,50)                          |
| $\sum L_{parede}$ | Soma da atenuação das paredes cruzadas pelo raio entre roteador e ponto (seção 3.4)  |
| $L_{laje,i}$      | Perda por laje: 15 dB para cada pavimento de diferença (não depende da banda)        |

Em 2,4 GHz a expressão se reduz à fórmula original do projeto,  $P_{tx} - 22\log_{10}(d) - \sum L_{parede} - L_{laje}$  (coeficiente 22 = expoente 2,2).

### 3.2 Catálogo de hardware

| Modelo                            | Potência base ( $P_{tx}$ ) | Ícone (desenhado por <b>CustomPainter</b> )                                  |
|-----------------------------------|----------------------------|------------------------------------------------------------------------------|
| Huawei AX3 Pro / AX3s             | 18 dBm                     | Roteador de mesa, corpo escuro, 4 antenas externas, LED verde                |
| TP-Link Deco (Mesh)               | 23 dBm                     | Torre/cilindro branco minimalista, sem antenas                               |
| Ubiquiti UniFi (AP Pro)           | 28 dBm                     | Disco de teto circular com anel de luz azul central                          |
| ZTE E2320 / E2620 (ZXHN, Wi-Fi 6) | 20 dBm                     | ONT/roteador branco de mesa, duas antenas externas inclinadas, faixa de LEDs |

A escolha do modelo é feita por uma **BottomSheet** ao tocar em **Adicionar Roteador**; o modelo selecionado também é o padrão para roteadores inseridos por toque na planta.

O ZTE E2320 foi cadastrado com potência (20 dBm) **acima do Huawei AX3** (18 dBm), refletindo o maior ganho/EIRP típico de um CPE Wi-Fi 6 com beamforming — na simulação, cobertura e alcance maiores que o Huawei nas mesmas condições de parede e distância.

### 3.3 Distância 3D e escala

Até a v7.0 a escala era fixa (45 px por metro). Na v8.0 cada pavimento tem a **largura real**  $W_m$  da planta, em metros (**FloorState.widthM**), e todo o cálculo é feito em **metros**, independentemente do tamanho da tela. As posições dos roteadores, pontos e paredes continuam guardadas como **frações** (0 a 1) da planta:

$$x_m = x \cdot W_m, \quad y_m = y \cdot \frac{W_m}{r}, \quad r = \frac{\text{largura}}{\text{altura}} \text{ (proporção da imagem)}$$

$$d_{3D} = \max \left( \sqrt{(\Delta x_m)^2 + (\Delta y_m)^2 + (\Delta a \cdot h)^2}, 1 \right)$$

Onde  $\Delta a$  é a diferença de andares (0 = mesmo andar) e  $h = 3$  m (`kFloorHeightM`).

**Calibração pela régua.** O usuário traça uma linha sobre uma medida conhecida (por exemplo, uma parede) e informa a distância real  $D$ . Com  $\ell$  o comprimento da linha em "larguras da planta",

$$\ell = \sqrt{(\Delta x)^2 + \left(\frac{\Delta y}{r}\right)^2}, \quad W_m = \frac{D}{\ell}$$

O novo  $W_m$  substitui o valor estimado e o mapa de calor é recalculado. Enquanto o pavimento não é calibrado, vale `defaultWidthM` da planta (8 m para a Casa 2 Quartos, 12 m para as plantas 01 e 02, 10 m para a 03 e para uploads, 20 m para o edifício) e o selo de escala mostra "(estimada)". Cada pavimento tem sua própria calibração.

### 3.4 Física de barreiras (ray-casting)

Cada planta tem uma lista de **segmentos de parede** (`WallSegment`) em coordenadas fracionárias, com uma atenuação em dB. Para cada célula da grade e cada roteador, o motor traça o segmento de reta entre eles e soma a atenuação de **todas as paredes que o raio cruza**.

O teste de interseção usa a orientação de pontos (caso geral):

```
bool _segmentsIntersect(Offset p1, Offset p2, Offset p3, Offset p4) {
    double orient(Offset a, Offset b, Offset c) =>
        (b.dx - a.dx) * (c.dy - a.dy) - (b.dy - a.dy) * (c.dx - a.dx);
    final o1 = orient(p1, p2, p3), o2 = orient(p1, p2, p4);
    final o3 = orient(p3, p4, p1), o4 = orient(p3, p4, p2);
    return ((o1 > 0) != (o2 > 0)) && ((o3 > 0) != (o4 > 0));
}
```

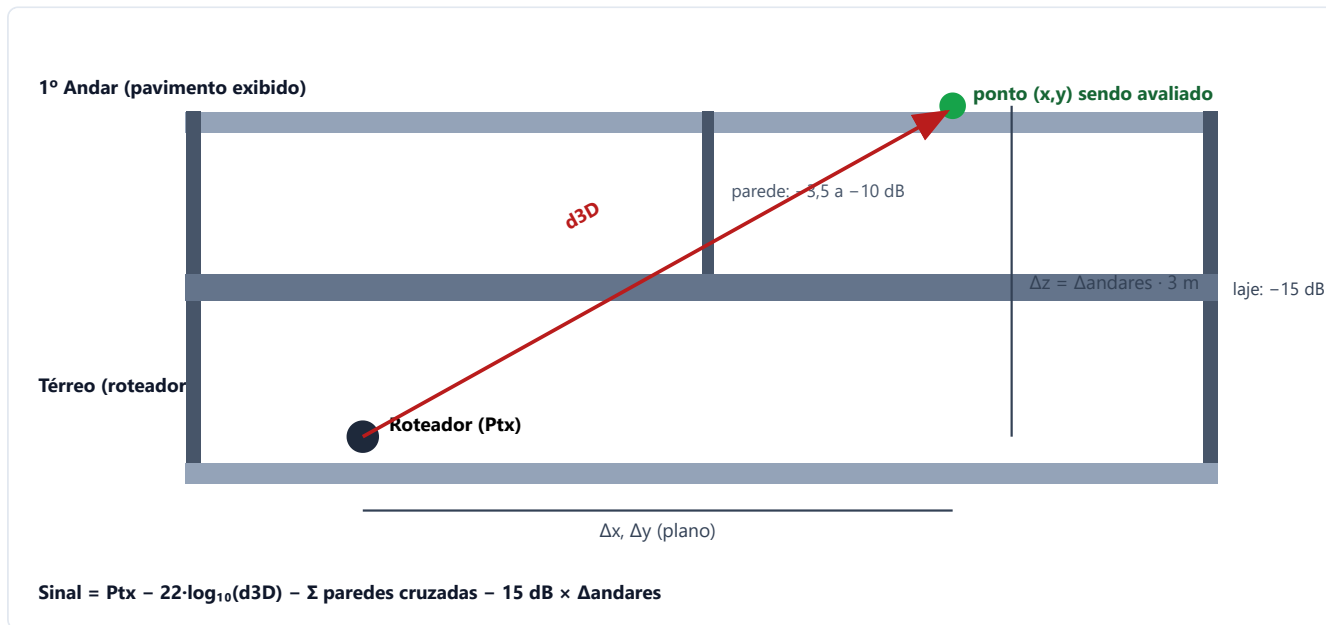
| Tipo de barreira                   | Atenuação usada (2.4 GHz)           | Onde aparece                                     |
|------------------------------------|-------------------------------------|--------------------------------------------------|
| Parede leve / divisória de varanda | 3,5 dB                              | Varandas e terraços (plantas da biblioteca)      |
| Parede interna padrão              | 6,0 dB (padrão)                     | Divisórias entre cômodos (plantas da biblioteca) |
| Parede pesada                      | 10,0 dB                             | Parede central do escritório                     |
| <b>Gesso / Drywall</b>             | 2 dB                                | Paredes desenhadas pelo usuário                  |
| <b>Tijolo cerâmico</b>             | 4 dB                                | Paredes desenhadas pelo usuário                  |
| <b>Vidro / Espelho</b>             | 5 dB                                | Paredes desenhadas pelo usuário                  |
| <b>Concreto armado / Laje</b>      | 12 a 15 dB (padrão 13,5; ajustável) | Paredes desenhadas pelo usuário                  |
| Laje de concreto entre andares     | 15 dB por andar                     | Entre pavimentos                                 |

As paredes de cada planta da biblioteca são **aproximações** desenhadas a partir da imagem (não medidas a laser). O usuário pode **desenhar paredes próprias** com a ferramenta *Parede* (seção 5.3), escolhendo o material; elas valem para qualquer planta, inclusive as enviadas do dispositivo, e somam-se às da biblioteca. As paredes usadas no cálculo de um andar são as do pavimento exibido. Nas bandas de 5 e 6 GHz a atenuação de cada parede é multiplicada por  $f_{par}(b)$ .

### 3.5 Arquitetura 2.5D multi-pavimento

Um **projeto** (`ProjectDef`) tem um ou mais **pavimentos** (`FloorDef`), e cada pavimento usa uma planta (`FloorPlanDef`).

- **Seletor de pavimento:** uma linha de chips (Térreo, 1º Andar, 2º Andar...) acima da planta, com o número de roteadores de cada andar.
- **Botão "+ Pavimento":** acrescenta andares ao projeto atual a partir de imagens do dispositivo.
- **Roteadores de outros andares** aparecem esmaecidos (opacidade 0,55, sem interação) no andar exibido, indicando a origem do sinal.
- **Suposição:** os pavimentos são empilhados sobre a mesma pegada, isto é, a mesma posição fracionária  $(x, y)$  em cada andar.



Propagação 2.5D: distância 3D, paredes e laje

### 3.6 Exemplos numéricos

| Caso                                                              | Cálculo                                       | Sinal    | Cor aproximada |
|-------------------------------------------------------------------|-----------------------------------------------|----------|----------------|
| Huawei (18 dBm), 4 m, mesmo andar, 1 parede de 6 dB               | $18 - 22 \log_{10}(4) - 6 = 18 - 13,25 - 6$   | -1,2 dBm | verde          |
| Mesmo Huawei, ponto logo acima (1 andar, $d_{3D} = 3 \text{ m}$ ) | $18 - 22 \log_{10}(3) - 15 = 18 - 10,50 - 15$ | -7,5 dBm | ciano/azulado  |
| UniFi (28 dBm), 10 m, mesmo andar, sem paredes                    | $28 - 22 \log_{10}(10) = 28 - 22$             | +6,0 dBm | verde/amarelo  |

### 3.7 Visualização do mapa de calor

**Amostragem.** A planta é dividida em uma grade de células de 8 px (no laudo, o mesmo motor renderiza uma grade proporcional em imagem de 1000 px de largura). Para cada célula avalia-se  $S(x, y)$  por meio de [SignalField.at](#) e converte-se em cor e transparência.

**Escala.** O sinal é normalizado por  $t = \text{clamp}((S + 30) / 56, 0, 1)$ , ou seja, -30 dBm mapeia para  $t = 0$  e +26 dBm para  $t = 1$  (próximo à potência máxima do catálogo).

| $t$  | Cor (paleta "jet")    |
|------|-----------------------|
| 0,00 | Azul profundo #1E3A8A |
| 0,18 | Azul #2563EB          |
| 0,38 | Ciano #06B6D4         |
| 0,58 | Verde #22C55E         |

| $t$  | Cor (paleta "jet") |
|------|--------------------|
| 0,74 | Amarelo #FACC15    |
| 0,88 | Laranja #FB923C    |
| 1,00 | Vermelho #EF4444   |

**Transparência.** A opacidade cresce suavemente (interpolação *smoothstep* entre  $t = 0,04$  e  $t = 0,55$ ) até o máximo configurável. O padrão é **48 %**; o usuário ajusta entre 15 % e 90 % em uma folha inferior (ícone de gota na barra superior). No Modo Diagnóstico (seção 5.4) a opacidade exibida é multiplicada por 1,25 (limite de 95 %). Regiões sem cobertura ficam transparentes, deixando a planta visível.

**Suavização.** A camada é desfocada (`ImageFilter.blur`, sigma 9) para remover o aspecto quadriculado.

**Legenda.** Os limiares são as constantes `kStrongDbm = 12` e `kWeakDbm = -10`: **Forte**  $\geq 12$  dBm; **Intermediário** de  $-10$  a  $12$  dBm; **Ruim**  $< -10$  dBm. A legenda fica sobre o mapa (fixa, sem acompanhar o zoom); no laudo aparece também uma faixa contínua de  $-30$  a  $+26$  dBm.

### 3.8 Interação

O mapa fica dentro de um `InteractiveViewer` (zoom de  $1\times$  a  $12\times$ ). Como o `InteractiveViewer` e os gestos de arrastar disputam o mesmo toque, toda a interação com os elementos do mapa usa **ponteiros crus** (`Listener`), que não entram na disputa de gestos; um segundo dedo cancela o desenho em curso e deixa o `InteractiveViewer` tratar o pinch. Detalhes na seção 5.3.

| Ação                                              | Comportamento                                                                                                                                       |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| Pinch com 2 dedos / roda do mouse / botões + e -  | Zoom e deslocamento do mapa; o botão "ajustar à tela" volta a $1\times$                                                                             |
| Ferramenta <i>Roteador</i> : tocar em área livre  | Insere um roteador do modelo atual naquele ponto                                                                                                    |
| Botão "Roteador" (barra inferior)                 | Abre a seleção de modelo e insere o roteador no centro (com pequeno deslocamento para não sobrepor)                                                 |
| Ferramenta <i>Roteador</i> : arrastar um roteador | Move o roteador dentro dos limites da planta; o mapa de calor é recalculado a cada quadro. Arrastar em área livre desloca o mapa (quando ampliado)  |
| Tocar em um roteador existente                    | Não cria um roteador duplicado                                                                                                                      |
| Botão de lixeira                                  | Remove todos os roteadores (de todos os andares)                                                                                                    |
| Anéis pulsantes                                   | Dois anéis expansivos por roteador (ciclo de 2,4 s) sugerem a emissão de sinal; roteadores e anéis mantêm tamanho constante na tela, mesmo com zoom |

### 3.9 Limitações do modelo

- É um modelo **simplicado** de perda de percurso; não modela multipercurso, difração, interferência entre roteadores nem a orientação das antenas. A frequência é considerada de forma **aproximada** (três bandas, seção 3.10) e os materiais são valores típicos, não medições.
- O nível "dBm" do simulador é uma escala **relativa** (potência de transmissão menos perdas, com o nível de referência a 1 m igual à potência do modelo); **não é** o RSSI que um aparelho mediria. Por isso o laudo mostra os dois lado a lado, sem compará-los numericamente (seção 5.7).
- A escala só é confiável depois de calibrada com a régua; sem isso vale a largura estimada da planta.
- As paredes das plantas da biblioteca são aproximações visuais; plantas enviadas pelo usuário só têm as paredes que ele desenhar.
- Em imagens em perspectiva 3D as paredes do cálculo são apenas indicativas.

- Os pavimentos são empilhados sobre a mesma pegada (mesma posição fracionária em cada andar), e a perda de laje é fixa (15 dB) em todas as bandas.

### 3.10 Simulação em 2.4 / 5 / 6 GHz

Um alternador de banda (chips **2.4 GHz** · **5 GHz** · **6 GHz**) refaz o mapa de calor na hora. A referência é 2.4 GHz; nas bandas mais altas o alcance diminui por três efeitos, todos parametrizados em [kRfBands](#) :

| Banda   | $\Delta_{ref}$ (dB, a 1 m) | $k$ (= 10·n) | $f_{par}$ (paredes) |
|---------|----------------------------|--------------|---------------------|
| 2.4 GHz | 0                          | 22           | 1,00                |
| 5 GHz   | -6,4                       | 24           | 1,35                |
| 6 GHz   | -7,9                       | 25           | 1,50                |

O ajuste  $\Delta_{ref}$  é a diferença de perda em espaço livre a 1 m,  $20\log_{10}(f / 2,4 \text{ GHz})$ , para 5,0 e 5,95 GHz. O expoente maior e a opacidade maior das paredes representam a pior propagação em obstáculos. Os valores são estimativas de engenharia para comparação relativa entre bandas, não medições. A banda escolhida é salva com o projeto e usada no laudo (que pode incluir várias bandas).

## 4. Módulo de Diagnóstico Mobile (Android Nativo)

### 4.1 Modos de operação

| Modo      | Condição                                                     | Dados              | Indicação na tela                                   |
|-----------|--------------------------------------------------------------|--------------------|-----------------------------------------------------|
| Nativo    | <code>window.NetFloorNative</code> existe (WebView do Shell) | Reais, do aparelho | Faixa verde: "Dados reais do aparelho Android."     |
| Simulação | Navegador comum ou PWA                                       | Fictícios          | Faixa amarela: "Modo simulação: dados fictícios..." |

A fonte de dados é escolhida por `DiagnosticsController`: começa em `SimulatedDiagnostics` e troca para `NativeBridgeDiagnostics` assim que a ponte é detectada, limpando os dados anteriores. Ambas implementam `DiagnosticsSource` (`ensurePermissions`, `scan`, `linkInfo`, `ping`).

### 4.2 Requisitos e permissões do Android

| Requisito                                      | Motivo                                                                                                                                  |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Permissão de <b>Localização</b> concedida      | O Android só devolve redes Wi-Fi e SSID/BSSID a apps com localização                                                                    |
| <b>GPS / Localização do aparelho ligada</b>    | Sem ela <code>getScanResults</code> retorna vazio a partir do Android 9                                                                 |
| <code>NEARBY_WIFI_DEVICES</code> (Android 13+) | Alternativa moderna de permissão para varredura                                                                                         |
| Limite de varreduras                           | O Android limita <code>startScan</code> (cerca de 4 a cada 2 minutos em primeiro plano); nesse caso retorna o último resultado em cache |

Se a permissão for negada, ou a localização estiver desligada, um cartão laranja explica o problema e oferece o botão **Conceder**.

### 4.3 Aba Espectro e Saúde do Canal

**Coleta.** O Kotlin chama `WifiManager.startScan()` e lê `scanResults`. Para cada rede são enviados: SSID, BSSID, frequência do canal primário, frequência central do bloco (`centerFreq0`), largura (20/40/80/160 MHz), nível em dBm e um indicador `connected` (comparação do BSSID com o da conexão atual). Redes fora de 2.4 GHz e 5 GHz (por exemplo 6 GHz) são descartadas da aba Espectro. Já a medição de campo (seção 5.5) rotula corretamente uma conexão em 6 GHz (frequência  $\geq 5925$  MHz).

**Relação canal  $\rightleftharpoons$  frequência.**

$$f_{2,4}(c) = 2407 + 5c \quad (c = 1..13; c = 14 \Rightarrow 2484 \text{ MHz}), \quad f_5(c) = 5000 + 5c$$

**Desenho.** `SpectrumPainter` desenha um **trapézio por rede**: a base tem a largura do canal, a altura é proporcional ao nível (eixo de  $-100$  a  $-30$  dBm) e o topo é 10 % mais estreito. O eixo X cobre 2396–2498 MHz (2.4 GHz) e 5160–5840 MHz (5 GHz). No 5 GHz o gráfico tem largura mínima de 980 px e rola horizontalmente para que todos os canais fiquem legíveis.

| Elemento       | Estilo                                                                                                                                       |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Rede conectada | Verde aceso ( <code>#00E676</code> ) com brilho, preenchimento, contorno, <b>barra sublinhada no eixo</b> e rótulo sublinhado com SSID e dBm |

| Elemento                  | Estilo                                                                                                                       |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Mesmo SSID em outra banda | Tratado como "sua rede": ex.: você está no 5 GHz e o 2.4 GHz do mesmo roteador aparece em destaque com o sufixo "- sua rede" |
| Redes vizinhas            | Cinza translúcido; rótulos só das 4 mais fortes, para evitar poluição                                                        |

Um cartão **Rede conectada** mostra SSID, BSSID, banda e canal, largura em MHz e nível em dBm. Se o Android ocultar o BSSID (sem permissão), o campo exibe "oculto pelo Android".

**Saúde do canal.** Para cada canal de 20 MHz calcula-se uma pontuação de interferência a partir das redes vizinhas. A sua própria rede (conectada e as de mesmo SSID) fica **de fora**.

$$o_{c,n} = \text{clamp}\left(\frac{\min(f_c + 10, h_{i_n}) - \max(f_c - 10, l_{o_n})}{20}, 0, 1\right), \quad s_n = \text{clamp}\left(\frac{RSSI_n + 95}{55}, 0, 1\right)$$

$$\text{score}_c = \sum_n o_{c,n} \cdot s_n$$

Onde  $[l_{o_n}, h_{i_n}]$  é o bloco de frequências ocupado pela rede  $n$  (centro  $\pm$  largura/2).

| Pontuação    | Classificação    |
|--------------|------------------|
| < 0,25       | <b>Excelente</b> |
| 0,25 a < 1,0 | <b>Bom</b>       |
| $\geq 1,0$   | <b>Ruim</b>      |

São avaliados os canais 1–13 (2.4 GHz) e 25 canais de 5 GHz (36–64, 100–144, 149–165). A tela lista os **3 canais recomendados** (menor pontuação) e uma tabela completa com canal, frequência, barra de interferência, número de redes sobrepostas e classificação.

**Atualização.** A varredura roda ao abrir a aba e a cada 30 s enquanto ela estiver visível; há também o botão **Escanear**.

## 4.4 Aba Sinal (walk-through)

O aplicativo lê o RSSI da conexão atual a cada **1 segundo** e mantém as últimas 60 amostras em um gráfico de linha (`fl_chart`). Linhas tracejadas de referência marcam  $-60$  dBm e  $-70$  dBm. A tela mostra o valor atual, SSID, banda e canal, além de mínimo, média e máximo da janela.

| RSSI              | Classificação |
|-------------------|---------------|
| $\geq -50$ dBm    | Excelente     |
| $-50$ a $-60$ dBm | Muito bom     |
| $-60$ a $-70$ dBm | Bom           |
| $-70$ a $-80$ dBm | Fraco         |
| $< -80$ dBm       | Muito fraco   |

Os botões **Pausar/Retomar** e **Limpar** controlam a coleta. O uso típico é caminhar pelo ambiente e observar as quedas do gráfico.

## 4.5 Aba Latência e Rede (ping duplo)

A cada segundo o aplicativo mede, **em paralelo**, a latência até:

1. **Gateway local:** IP do roteador obtido de `DhcpInfo.gateway` (atualizado a cada 5 ciclos).

## 2. DNS da Internet: 8.8.8.8.

| Indicador                | Origem                                                               |
|--------------------------|----------------------------------------------------------------------|
| Perda de pacotes (%)     | perdidos / enviados × 100, por alvo                                  |
| Latência DNS (ms)        | Última medição e média                                               |
| Latência do gateway (ms) | Última medição, média e IP                                           |
| Velocidade PHY (Mbps)    | <code>WifiInfo.getLinkSpeed()</code> (taxa negociada com o roteador) |

**Implementação do ping no Android.** O Kotlin executa `/system/bin/ping -c 1 -W 2 <host>` e extrai o valor `time=` da saída. Se o binário não puder ser executado, usa-se um **fallback TCP**: mede-se o tempo de `connect` nas portas 53 e 80 (conexão recusada também prova alcance). O nome do host é validado antes da execução.

**Interpretação.** Se só o DNS piora, o problema tende a estar fora de casa; se o gateway também piora, o problema é na rede local (Wi-Fi). Pacotes perdidos aparecem como lacunas no gráfico.

## 4.6 Controlador de diagnóstico

`DiagnosticsController` executa apenas o trabalho da **aba visível**, poupando bateria e respeitando os limites do Android.

| Aba      | Temporizador                     | Proteções                                              |
|----------|----------------------------------|--------------------------------------------------------|
| Espectro | Varredura imediata + a cada 30 s | Não inicia nova varredura enquanto outra roda          |
| Sinal    | A cada 1 s (se "Retomar")        | Ignora ciclos enquanto a leitura anterior não terminou |
| Latência | A cada 1 s (se "Retomar")        | Idem; os dois pings rodam com <code>Future.wait</code> |

Ao trocar de aba ou de tela, os temporizadores são cancelados e recriados conforme necessário. As amostras permanecem guardadas quando o usuário alterna entre abas.

## 4.7 Modo simulação

`SimulatedDiagnostics` gera 13 redes fictícias (2.4 e 5 GHz, larguras variadas, uma rede conectada e o 2.4 GHz de mesmo nome), um RSSI que oscila como uma caminhada, IP de gateway `192.168.0.1`, latências plausíveis e ~3 % de perda. Serve para demonstrar a interface em qualquer navegador.

## 4.8 Estado de validação

| Item                                       | Situação (na data desta revisão)                                                                                                   |
|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Modo simulação (3 abas)                    | Verificado no navegador                                                                                                            |
| Ponte com dados no formato nativo          | Verificada injetando uma ponte simulada no navegador                                                                               |
| Dados reais no Android                     | <b>Validado em aparelho real</b> (Xiaomi Redmi Note 14 Pro): espectro 2.4/5 GHz, gateway <code>192.168.100.1</code> , PHY 390 Mbps |
| Seleção múltipla de imagens no Shell 3.1.0 | Compilado; ainda não validado em aparelho                                                                                          |
| Testes automatizados                       | Não há (o app usa <code>dart:js_interop</code> , que não executa na VM de testes)                                                  |

## 5. NetFloor Enterprise (v8.0)

### 5.1 Visão geral

| Recurso               | Resumo                                                                                                | Seção     |
|-----------------------|-------------------------------------------------------------------------------------------------------|-----------|
| Ferramentas do mapa   | Roteador, <b>Parede</b> (com materiais), <b>Régua</b> (calibração de escala) e <b>Ponto</b> (medição) | 5.3       |
| Zoom e deslocamento   | <b>InteractiveViewer</b> com pinch, roda do mouse e botões                                            | 5.3       |
| Modos de visualização | <b>Apresentação</b> (escuro/moderno) e <b>Diagnóstico</b> (alto contraste)                            | 5.4       |
| Medição de campo      | RSSI, PHY, ping ao gateway e à Internet, perda de pacotes                                             | 5.5       |
| Projetos offline      | Salvamento automático em IndexedDB; exportar/importar <b>.json</b>                                    | 5.6       |
| Laudo de vistoria     | PDF com logo, mapas de calor por banda, tabelas e assinatura do cliente                               | 5.7 e 5.8 |
| Entrega de arquivos   | Download no navegador; salvar/compartilhar no Shell 3.2+                                              | 5.9       |

A banda de operação (2.4/5/6 GHz) e a física de paredes/escala estão nas seções 3.3, 3.4 e 3.10.

### 5.2 Fluxo de trabalho típico em campo

1. Abrir uma planta da biblioteca ou **carregar a planta do cliente** (foto/imagem).
2. Com a **Régua**, medir uma parede conhecida e informar a distância real (calibra a escala).
3. Com **Parede**, desenhar as paredes reais escolhendo o material.
4. Posicionar os roteadores e comparar as bandas 2.4/5/6 GHz.
5. Marcar **pontos** nos cômodos críticos e, no local, tocar em **Medir agora** (ou usar *Registrar medição* na aba Diagnóstico).
6. Abrir o **Laudo**, preencher os dados, coletar a **assinatura** do cliente e gerar o PDF.

Tudo é salvo automaticamente no aparelho; o projeto pode ser retomado depois e exportado em **.json**. **Atenção:** os dados ficam offline, mas o *aplicativo* ainda é carregado do GitHub Pages ( `--pwa-strategy=none`, base do OTA), então é preciso ter internet ao abrir o app (ver seção 8).

### 5.3 Ferramentas do mapa e gestos

| Ferramenta      | Gesto                                      | Resultado                                                                                                                                                                                                  |
|-----------------|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Roteador</b> | Toque em área livre; arrastar um roteador  | Insere / move roteadores (seção 3.8)                                                                                                                                                                       |
| <b>Parede</b>   | Arrastar no mapa; tocar em uma parede      | Traça uma parede (com <b>encaixe</b> em horizontal/vertical a menos de ~4° e nos cantos de outras paredes); tocar abre a folha para trocar o material, ajustar a perda do concreto (12 a 15 dB) ou remover |
| <b>Régua</b>    | Arrastar sobre uma medida conhecida        | Mostra "≈ X m" durante o arraste e abre o diálogo de distância real; confirma e recalcula a escala                                                                                                         |
| <b>Ponto</b>    | Tocar no mapa; tocar em um ponto; arrastar | Cria/edita/move um ponto de medição (nome, previsão, "Medir agora", remover)                                                                                                                               |

Materiais de parede (cores no mapa): Gesso/Drywall -2 dB (verde), Tijolo cerâmico -4 dB (âmbar), Vidro/Espelho -5 dB (ciano), Concreto armado/Laje -12 a -15 dB (vermelho). As paredes aproximadas da biblioteca aparecem tracejadas em cinza quando a ferramenta *Parede* está ativa.

**Zoom e gestos.** Dois dedos fazem pinch/pan (o `InteractiveViewer` cuida disso). O deslocamento com um dedo só fica ativo nas ferramentas *Roteador* e *Ponto*, e apenas quando o toque não começa sobre um marcador. Nas ferramentas de desenho, um dedo desenha e dois dedos navegam. Os traços, marcadores e anéis compensam o zoom (`1/zoom`) para manter tamanho constante na tela.

**Por que `Listener`?** O reconhecedor de escala do `InteractiveViewer` aceita o gesto (a partir de ~18 px) antes do `onPanUpdate` de um filho (~36 px), o que impediria arrastar roteadores. Ponteiros crus (`onPointerDown/Move/Up`) recebem todos os eventos sem disputar a arena de gestos; a lógica de toque (limite de 8 px de movimento e 600 ms) e de arraste é própria. Cancelar por segundo dedo evita que um pinch deixe uma parede "pela metade".

## 5.4 Modos de visualização

| Modo                            | Aparência                                                                                                                                                                    | Uso                            |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| <b>Apresentação</b><br>(padrão) | Tema escuro ( <code>#0B1020</code> ), cartões arredondados, planta com filtro que inverte a luminosidade e preserva os matizes                                               | Mostrar a simulação ao cliente |
| <b>Diagnóstico</b>              | Alto contraste: fundo branco, barra preta, chips amarelos, contornos pretos, texto $\geq 1,08\times$ ; planta em tons de cinza com contraste e mapa de calor 25 % mais opaco | Uso externo sob sol forte      |

O botão de contraste na barra do simulador alterna os modos; a escolha é salva (`settings.viewMode`). O filtro só afeta a **exibição**: o laudo em PDF sempre usa a planta original, própria para impressão.

## 5.5 Pontos de medição e medição de campo

`DiagnosticsController.captureSnapshot` (5 pares de ping por padrão) devolve um `Measurement`:

| Campo                                                                                                  | Origem                                                                                      |
|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <code>rsssi</code> , <code>linkMbps</code> (PHY), <code>ssid</code> , <code>bssid</code> , banda/canal | <code>linkInfo</code> (WifiInfo)                                                            |
| <code>gwAvgMs</code> , <code>gwLossPct</code>                                                          | Ping ao gateway (DHCP)                                                                      |
| <code>netAvgMs</code> , <code>netLossPct</code>                                                        | Ping ao DNS público <code>8.8.8.8</code>                                                    |
| <code>native</code>                                                                                    | <code>true</code> só quando a fonte é o Shell Android; <code>false</code> = <b>simulado</b> |

Uma medição pode ser vinculada a um **ponto** (o ponto guarda a última) e sempre entra no **histórico do projeto** (`logs`), que é salvo e exportado. No mapa, um ponto medido tem uma bolinha branca no centro; a cor do ponto é a classe prevista (verde/âmbar/vermelho). Fora do Shell os dados são simulados e assim marcados no histórico e no laudo (aviso em vermelho), para nunca serem confundidos com a rede real do cliente.

## 5.6 Persistência offline-first e arquivo `.json`

**Armazenamento local.** `AppStore` usa `sembast_web` (IndexedDB) com o banco `netfloor_v8`:

| Store                   | Chave                      | Conteúdo                                           |
|-------------------------|----------------------------|----------------------------------------------------|
| <code>workspaces</code> | id do projeto              | Documento JSON do projeto (sem bytes de imagem)    |
| <code>images</code>     | id da planta personalizada | Imagem em base64                                   |
| <code>settings</code>   | nome                       | <code>viewMode</code> , <code>lastWorkspace</code> |

O salvamento é automático, 0,8 s após a última alteração, e imediato quando a página é escondida (`visibilitychange`). Ao abrir o app, o último projeto é restaurado. A folha **Projetos e plantas** lista os projetos salvos (com exclusão confirmada), as plantas da biblioteca, o upload de plantas e a importação. Se o IndexedDB estiver indisponível (por

exemplo, aba anônima), o app segue funcionando sem persistir e avisa na folha de projetos. A WebView do Shell mantém o DOM Storage ligado por padrão, então o armazenamento também funciona no aplicativo Android.

**Exportar/importar.** *Exportar projeto (.json)* gera `<nome>.netfloor.json` (indentado, UTF-8) com **tudo**: plantas personalizadas embutidas em base64, escala e calibração, paredes, roteadores, pontos, medições, dados do laudo (inclusive logo e assinatura) e banda. Plantas da biblioteca entram só como referência (`planId`), o que mantém o arquivo leve (um projeto típico tem ~25 KB). *Importar* cria um **novo** projeto local (novo id, novos ids de imagem), sem sobrescrever o existente.

```
{
  "format": "netfloor-project", "schema": 1, "appVersion": "8.0",
  "id": "...", "name": "...", "updatedAt": "...", "projectId": "casa_2q",
  "band": "ghz5", "model": "huaweiAx3", "heatOpacity": 0.48,
  "floors": [ { "label": "Térreo", "planId": "casa_2q", "custom": false,
    "widthM": 13.3, "calibrated": true,
    "walls": [ { "ax": 0.27, "ay": 0.42, "bx": 0.74, "by": 0.42, "db": 2, "mat": "drywall" } ] },
  "routers": [ { "id": "r0", "x": 0.67, "y": 0.63, "floor": 0, "model": "huaweiAx3" } ],
  "points": [ { "id": "p...", "name": "P1", "x": 0.24, "y": 0.67, "floor": 0, "measured": { } } ],
  "logs": [ ], "report": { }
}
```

`applyJson` valida o formato (`format = netfloor-project`), a existência das plantas e das imagens e rejeita arquivos inválidos com mensagem clara.

## 5.7 Laudo de vistoria em PDF

A tela **Laudo de vistoria** (ícone de documento na barra do simulador) reúne os dados do laudo, que são salvos com o projeto:

| Campo                                  | Uso no PDF                                                |
|----------------------------------------|-----------------------------------------------------------|
| Técnico responsável, Empresa/ISP       | Cabeçalho, identificação e bloco de assinatura do técnico |
| Logo da empresa (PNG/JPG/WEBP enviado) | Canto superior esquerdo da capa                           |
| Cliente, Endereço, Data                | Identificação (a data padrão é o dia atual)               |
| Bandas com mapa de calor               | Uma página por pavimento e banda escolhida                |
| Observações do técnico                 | Caixa de texto livre                                      |
| Assinatura do cliente                  | Bloco final (seção 5.8)                                   |

### Conteúdo do PDF (A4).

- Capa e resumo:** identificação, resumo da simulação (pavimentos, roteadores, bandas, se a escala foi calibrada, pontos críticos, medições) e tabela de roteadores (modelo, andar, potência, posição em metros).
- Mapas de calor** (uma página por pavimento e banda): planta + calor + paredes desenhadas + roteadores numerados (R1, R2...) + pontos com o valor previsto + barra de escala; **legenda detalhada de dBm** (faixa contínua de -30 a +26 dBm e as classes Forte/Intermediário/Ruim) e o resumo de cobertura do andar (% forte, intermediária e ruim, pior ponto com coordenadas em metros).
- Diagnóstico e pontos críticos:** cobertura por pavimento e banda; tabela **previsto x medido** (ponto, andar, nível simulado, classe, RSSI, PHY, ping ao gateway, ping à Internet, perda, status); registro de diagnósticos de campo; observações do técnico; assinaturas.

**Status dos pontos.** *CRÍTICO:* classe Ruim ( $< -10$  dBm simulados),  $\text{RSSI} \leq -75$  dBm, perda  $\geq 5$  % ou ping ao gateway  $> 30$  ms. *Atenção:* classe Intermediária,  $\text{RSSI} \leq -67$  dBm, alguma perda ou ping ao gateway  $> 15$  ms. Caso contrário, OK. O PDF traz uma nota explicando que o **nível simulado (escala relativa)** e o **RSSI medido não são numericamente comparáveis**, e um aviso em destaque quando existem medições simuladas.

## Como é gerado.

| Etapa                | Implementação                                                                                                                                                                                                                                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Renderização do mapa | <code>renderFloorRaster</code> : <code>PictureRecorder</code> + o mesmo <code>HeatmapPainter</code> / <code>SignalField</code> do simulador, com desfoque por <code>saveLayer</code> ( <code>ImageFilter.blur</code> ), roteadores por <code>RouterDevicePainter</code> , pontos por <code>PointMarkerPainter</code> ; largura de 1000 px |
| Compressão           | A imagem é codificada em <b>JPEG</b> (qualidade 88, <code>package:image</code> ) e incorporada ao PDF <b>sem recompressão</b> ; um laudo típico com duas bandas fica em torno de 1 MB                                                                                                                                                     |
| Documento            | Biblioteca <code>pdf</code> (100 % Dart, roda no navegador e na WebView): <code>MultiPage</code> / <code>Page</code> A4, tabelas com <code>TableHelper</code> , rodapé com paginação                                                                                                                                                      |
| Fonte                | Roboto Regular/Bold/Italic embutida em <code>assets/fonts/</code> (acentos, "≥", "–")                                                                                                                                                                                                                                                     |

O laudo é gerado localmente; nada é enviado a servidores.

## 5.8 Assinatura digital do cliente

O botão **Coletar assinatura** abre uma tela cheia com um quadro branco (`SignaturePadPage`): o cliente assina com o dedo (ou mouse). O traço é capturado por ponteiros crus, suavizado com curvas quadráticas pelos pontos médios, e exportado como **PNG recortado** ao retângulo da assinatura (margem de 14 px, ×2,5) sobre fundo branco. A imagem, o nome do signatário e a data/hora ficam no projeto e aparecem no PDF ("Assinado digitalmente em dd/mm/aaaa hh:mm"). É uma assinatura **eletrônica simples** (imagem do traço), sem certificado digital; não substitui uma assinatura com validade jurídica ICP-Brasil.

## 5.9 Entrega de arquivos (PDF e `.json`)

| Ambiente              | Comportamento                                                                                                                                                                                                                                        |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Navegador / PWA       | Download por <code>Blob</code> + <code>&lt;a download&gt;</code> ( <code>FileIO._browserDownload</code> )                                                                                                                                            |
| Shell Android ≥ 3.2.0 | <code>FileIO.deliver</code> envia o arquivo pela ponte ( <code>saveFile</code> , em pedaços de base64): <b>Gerar laudo em PDF</b> salva em <code>Downloads/NetFloor</code> ; <b>Gerar e compartilhar</b> abre a folha de compartilhamento do Android |
| Shell Android < 3.2.0 | O app mostra "Atualize o app NetFloor Shell (3.2 ou superior)"                                                                                                                                                                                       |

O Kotlin usa `MediaStore.Downloads` (Android 10+), sem pedir permissão de armazenamento, e `FileProvider` (`<appId>.fileprovider`, `cache-path shared/`) para o compartilhamento. Nomes de arquivo são sanitizados.

## 5.10 Estado de validação dos recursos da v8.0

| Recurso                                                                                                     | Situação                                                                           |
|-------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Ferramentas, calibração, materiais, banda, zoom, modos de visualização                                      | Validados no navegador (viewport de celular)                                       |
| Persistência (recarregar a página), exportar e importar <code>.json</code> , upload de planta personalizada | Validados no navegador, incluindo imagem embutida                                  |
| Laudo em PDF com logo, duas bandas, medição simulada e assinatura                                           | Gerado no navegador e conferido página a página                                    |
| <code>saveFile</code> (Downloads e compartilhamento) no Shell 3.2.0                                         | <b>Compilado; ainda não validado em aparelho</b> (sem <code>adb</code> disponível) |

| Recurso                                     | Situação                                                                                                         |
|---------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Medição real (RSSI/ping) vinculada a pontos | Depende do Shell; a leitura real já foi validada na v6.x, o vínculo com pontos ainda não foi testado em aparelho |

## 6. Gestão de Plantas e Mídia

### 6.1 Modelo de dados

| Classe       | Campos principais                                                                                        | Papel                                                                         |
|--------------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| ProjectDef   | id, name, subtitle, floors, isCustom                                                                     | Um projeto na biblioteca (casa, prédio, upload)                               |
| FloorDef     | label, plan                                                                                              | Um pavimento (Térreo, 1º Andar...) da biblioteca                              |
| FloorState   | label, plan, widthM, calibrated, userWalls                                                               | Pavimento <b>em uso</b> : planta + escala real + paredes desenhadas (mutável) |
| FloorPlanDef | id, name, subtitle, imageUrl?, assetPath?, memoryBytes?, aspectRatio, defaultWidthM, rooms, wallSegments | Uma planta: aparência + física                                                |
| RoomDef      | label, rectFrac                                                                                          | Cômodo do desenho vetorial                                                    |
| WallSegment  | a, b, attenuationDb, material?                                                                           | Parede para o ray-casting ( <b>material</b> só nas desenhadas pelo usuário)   |
| RouterNode   | id, frac, floor, model                                                                                   | Roteador posicionado                                                          |
| MeasurePoint | id, name, frac, floor, measured?                                                                         | Ponto de medição                                                              |
| Measurement  | time, native, rssi, linkMbps, gwAvgMs, netAvgMs, *LossPct, pointId? ...                                  | Medição de campo                                                              |
| ReportInfo   | técnico, empresa, cliente, endereço, data, notas, logo, signature, signedAt                              | Dados do laudo                                                                |

### 6.2 Cadeia de resolução da imagem

A planta de fundo é resolvida nesta ordem, com queda automática para a próxima opção em caso de falha:

1. **Memória** ( `memoryBytes` → `Image.memory` ): plantas enviadas pelo usuário.
2. **Rede** ( `imageUrl` → `Image.network` , com indicador de carregamento): arquivo em [raw.githubusercontent.com](https://raw.githubusercontent.com).
3. **Asset** ( `assetPath` → `Image.asset` ): arquivo embutido no app.
4. **Desenho vetorial** ( `rooms` → `FloorPlanPainter` ): piso texturizado, paredes espessas e silhuetas de móveis.

Isso permite deixar a **vaga** de uma planta reservada: enquanto o PNG não existir, o app mostra o desenho vetorial provisório; ao colocar o arquivo com o nome esperado em `assets/floorplans/`, a imagem passa a ser usada.

### 6.3 Biblioteca atual

| Projeto                                        | Pavimentos | Fonte visual                           | Situação                                                                                                                                                                                  |
|------------------------------------------------|------------|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Casa Térrea 2 Quartos ( <code>casa_2q</code> ) | 1          | <code>casa_2q.png</code> (736×1105 px) | Imagem limpa, com paredes aproximadas                                                                                                                                                     |
| Casa 2 Pavimentos (Referência)                 | 2          | Desenho vetorial próprio               | Modelo de referência para casas de dois andares (corte esquemático: sala/escada/sala de TV no térreo, sala com varanda/quarto no 1º andar) — permanentemente vetorial, sem foto associada |

| Projeto                             | Pavimentos | Fonte visual                                       | Situação                                                                       |
|-------------------------------------|------------|----------------------------------------------------|--------------------------------------------------------------------------------|
| Planta 01 — Apartamento 3 Quartos   | 1          | <code>planta_01.png</code> (aguardando)            | Desenho vetorial provisório                                                    |
| Planta 02 — Apartamento Open Space  | 1          | <code>planta_02.png</code> (aguardando)            | Desenho vetorial provisório                                                    |
| Planta 03 — Apartamento com Terraço | 1          | <code>planta_03.png</code> (aguardando)            | Desenho vetorial provisório                                                    |
| Planta 04 — Apartamento 3 Suítes    | 1          | <code>planta_04.png</code> (aguardando)            | Desenho vetorial provisório (cozinha, sala de jantar, sala de estar, 3 suítes) |
| Edifício Corporativo                | 3          | <code>edificio_corporativo.png</code> (aguardando) | Planta vetorial repetida nos 3 andares (Open Space, Reunião, Diretoria, Copa)  |

## 6.4 Como adicionar uma nova planta

1. Confirme que a imagem é **limpa** (sem marca d'água de terceiros e com direito de uso).
2. Copie o arquivo para `netfloor/assets/floorplans/` (o `pubspec.yaml` já inclui a pasta inteira).
3. No `main.dart`, crie um `FloorPlanDef` com `assetPath`, `imageUrl` (URL `raw.githubusercontent.com` após publicar no `main`) e `aspectRatio = largura / altura` da imagem.
4. Estime as paredes internas em coordenadas fracionárias (0 a 1) e cadastre os `WallSegment` (3,5 / 6 / 10 dB conforme a parede) e a largura real estimada em `defaultWidthM`. Se a planta for nova na biblioteca, inclua-a também em `kBuiltInPlans` (necessário para reabrir projetos salvos e importados).
5. Inclua a planta em um `ProjectDef` na lista `kProjectLibrary`.
6. Publique (seção 7): commit no `main`, build web e deploy no `gh-pages`.

## 6.5 Upload em memória (`Uint8List`)

O botão **Carregar plantas do dispositivo** (biblioteca) e o chip + **Pavimento** aceitam **uma ou várias** imagens (PNG, JPG, WEBP):

- A leitura usa `FilePicker.pickFiles(type: FileType.custom, allowedExtensions: [...])` e `PlatformFile.readAsBytes()`, obtendo um `Uint8List`. **Não há `dart:io`**, portanto a mesma lógica funciona em Web/PWA, no Shell Android e no desktop. (No `file_picker` 13 o parâmetro antigo `withData: true` não existe mais; os bytes vêm de `readAsBytes()`.)
- As dimensões são obtidas com `instantiateImageCodec`, definindo o `aspectRatio` sem esticar a imagem.
- Cada imagem vira um pavimento, na ordem de seleção (Térreo, 1º Andar...). No botão da biblioteca cria-se um **projeto novo**; no chip + **Pavimento** os andares são **acrescentados** ao projeto atual.
- A planta é guardada no armazenamento local (store `images`) e vai embutida no `.json` exportado (seção 5.6). Não tem paredes mapeadas: o usuário as desenha com a ferramenta *Parede* e calibra a escala com a *Régua* (seção 5.3).
- No Shell Android, o `<input type="file">` da página é atendido pelo seletor nativo (`setOnShowFileSelector`), com suporte a seleção múltipla a partir do Shell 3.1.0.

## 6.6 Política de conteúdo e licenças

- Somente imagens **limpas**, próprias ou licenciadas, devem entrar na biblioteca.
- Imagens com marca d'água de terceiros (por exemplo, prévias de bancos de imagens pagos ou de sites de projetos) **não** são incluídas nem têm a marca removida.

- Vagas de plantas cujo arquivo ainda não foi fornecido usam desenho vetorial próprio.
- Quando o usuário envia uma imagem de referência (ex.: um render de terceiros) para pedir uma planta "no estilo" dela, o app **não reproduz a imagem**: um novo `FloorPlanDef` vetorial é desenhado do zero, inspirado apenas na disposição geral dos cômodos (como "Casa 2 Pavimentos" e "Planta 04", v8.1).

## 6.7 Histórico de alterações

| Versão | Data       | Alterações                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0    | 14/09/2026 | Protótipo: duas plantas desenhadas em código (Casa e Apartamento), mapa de calor em 3 faixas (azul/verde/vermelho), toque para adicionar e arraste de roteadores                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 1.1    | 15/09/2026 | Planta real como fundo; gradiente contínuo "jet" com desfoque; ícone de roteador; anéis pulsantes; correção do repaint do mapa de calor; publicação no GitHub Pages; primeiro APK nativo (release v1.0.0)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 2.0    | 15/09/2026 | Catálogo de roteadores (18/23/28 dBm); biblioteca de plantas; fórmula $P_{tx} - 22\log_{10}(d)$ ; <b>arquitetura OTA</b> (site + Shell WebView, release v2.0.0)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 4.0    | 15/09/2026 | Plantas ilustradas (piso, móveis, paredes espessas); <b>ray-casting de paredes</b> ; overlay do calor a ~48 %; <code>Image.network</code> com fallback                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 5.1    | 16/09/2026 | Plantas reais como assets (Casa 2 Quartos com correção de espelhamento, Casa 3 Quartos e Apartamento 2 Quartos, este com marca d'água de terceiros aceita pelo usuário na época)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 6.0    | 18/09/2026 | Upload de plantas em memória; <b>NetFloor Diagnostic</b> (espectro, sinal, latência); ponte JS + Kotlin; Shell 3.0.0; assets movidos para <code>assets/floorplans/</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 6.1    | 18/09/2026 | Destaque e sublinhado da rede conectada e da rede de mesmo SSID em outra banda; saúde do canal desconsidera a própria rede                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 7.0    | 19/09/2026 | <b>Projetos com vários pavimentos (2.5D)</b> , distância 3D e perda de laje de 15 dB; posições dos roteadores em frações; upload em lote; novas plantas limpas; cartão "Rede conectada"; Shell 3.1.0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 7.0.1  | 19/09/2026 | <b>Limpeza da biblioteca</b> : remoção do projeto Sobrado (térreo e 1º andar) e do arquivo <code>sobrado_1andar.png</code> ; vagas <code>planta_01..03</code> com desenho provisório                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 8.0    | 21/09/2026 | <b>NetFloor Enterprise</b> : laudo de vistoria em PDF com logo, mapas por banda, tabelas previsto × medido e <b>assinatura digital</b> ; <code>InteractiveViewer</code> (zoom/pan); <b>calibração de escala por régua</b> (o cálculo passa a ser em metros reais); <b>materiais de parede</b> e ferramenta de desenho; <b>2.4 / 5 / 6 GHz</b> ; pontos e <b>medição de campo</b> (RSSI, PHY, ping duplo, perda); modos <b>Apresentação</b> e <b>Diagnóstico</b> ; <b>persistência offline</b> (IndexedDB) e exportar/importar <code>.json</code> ; Shell 3.2.0 ( <code>saveFile</code> , compartilhamento, seletor <code>.json</code> ); a constante <code>kPixelsPerMeter</code> deixa de existir |
| 8.1    | 25/09/2026 | <b>Biblioteca</b> : novo projeto "Casa 2 Pavimentos (Referência)" (2 andares, desenho vetorial próprio, modelo para simulações em casas de dois pisos) e "Planta 04 — Apartamento 3 Suítes"; heurísticas de mobília do desenho vetorial ganham mesa de jantar ( <code>jantar</code> ) e escada ( <code>escada</code> ). <b>Catálogo</b> : novo roteador <b>ZTE E2320 / E2620 (ZXHN, Wi-Fi 6)</b> , 20 dBm (acima do Huawei AX3), com ícone próprio (corpo branco, duas antenas externas)                                                                                                                                                                                                           |

### Remoções relacionadas a marca d'água e conteúdo.

| Item                                                                                                             | Situação                                            |
|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| <code>Apartamento 2 quartos.jpg</code> (marca <code>@opedreiro</code> )                                          | Removido do código, da biblioteca e do site na v7.0 |
| Antigas Casa 2 Quartos e Casa 3 Quartos                                                                          | Substituídas pela nova biblioteca na v7.0           |
| Sobrados ( <code>sobrado_1andar.png</code> e desenho do térreo)                                                  | Removidos na v7.0.1                                 |
| Imagens com marca de terceiros enviadas depois ( <code>montesuacasa.com.br</code> , <code>depositphotos</code> ) | <b>Não incluídas</b> no projeto                     |

**Observação importante:** a remoção é da **biblioteca, do código e do site publicado**. Os arquivos antigos continuam recuperáveis no **histórico de commits do Git** do repositório. Apagá-los de vez exige reescrever o histórico (`git filter-repo` e `push --force`), o que só deve ser feito com pedido explícito.

## 7. Guia de Instalação, Build e CI/CD

### 7.1 Ambiente de desenvolvimento (Windows 11)

| Componente       | Local / versão                                                                                                                                                                                          |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Flutter SDK      | <code>C:\flutter</code> — 3.47.4 stable (Dart 3.13.3); <code>C:\flutter\bin</code> no PATH                                                                                                              |
| JDK              | Eclipse Temurin 17.0.20 — <code>JAVA_HOME = C:\Program Files\Eclipse Adoptium\jdk-17.0.20.101-hotspot</code>                                                                                            |
| Android SDK      | <code>C:\Android\sdk</code> — <code>ANDROID_HOME / ANDROID_SDK_ROOT</code> ; plataformas 35 e 36; build-tools 28.0.3, 35.0.0 e 36.0.0; <code>platform-tools</code> e <code>cmdline-tools</code> no PATH |
| Git e GitHub CLI | <code>git</code> e <code>gh</code> autenticado na conta do projeto                                                                                                                                      |
| Navegador        | Chrome/Edge para desenvolvimento web                                                                                                                                                                    |

### 7.2 Dependências (pubspec.yaml)

#### App web (netfloor)

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.6
  file_picker: ^13.1.0 # upload de plantas/Logo/.json em memória (bytes), sem dart:io
  fl_chart: ^1.2.0 # gráficos de linha (sinal e latência)
  pdf: ^3.13.1 # laudo de vistoria em PDF (100 % Dart)
  image: ^4.10.1 # codificação JPEG dos mapas de calor do laudo
  web: ^1.1.1 # download de arquivos (Blob) e visibilitychange
  sembast: ^3.8.11 # banco NoSQL local
  sembast_web: ^2.4.6 # ... sobre IndexedDB (offline-first)

dev_dependencies:
  flutter_lints: ^3.0.0

flutter:
  uses-material-design: true
  assets:
    - assets/floorplans/
    - assets/fonts/ # Roboto (Regular, Bold, Italic) para o PDF
```

#### Shell Android (netfloor\_shell)

```
version: 3.2.0+5
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  webview_flutter: ^4.10.0
  webview_flutter_android: ^4.14.1
  permission_handler: 12.0.1 # a 13.x exige compileSdk 37 (não suportado pelo Gradle atual)
  file_picker: ^13.1.0 # seletor de arquivos para o <input type="file">
```

### 7.3 Permissões do Android (AndroidManifest.xml do Shell)

| Permissão             | Uso                             |
|-----------------------|---------------------------------|
| <code>INTERNET</code> | Carregar o app web e as imagens |

| Permissão                                                               | Uso                                                                         |
|-------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>ACCESS_NETWORK_STATE</code>                                       | Estado da conectividade                                                     |
| <code>ACCESS_WIFI_STATE</code>                                          | Ler <code>WifiInfo</code> , <code>DhcpInfo</code> e resultados de varredura |
| <code>CHANGE_WIFI_STATE</code>                                          | Solicitar <code>startScan()</code>                                          |
| <code>ACCESS_FINE_LOCATION</code> , <code>ACCESS_COARSE_LOCATION</code> | Exigidas pelo Android para varredura Wi-Fi e leitura de SSID/BSSID          |
| <code>NEARBY_WIFI_DEVICES</code>                                        | Permissão de dispositivos Wi-Fi próximos (Android 13+)                      |

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
<uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.NEARBY_WIFI_DEVICES"/>
```

Além das permissões, o manifesto do Shell 3.2.0 declara um `FileProvider` (`androidx.core.content.FileProvider`, autoridade `${applicationId}.fileprovider`, caminhos em `res/xml/file_paths.xml`: `cache-path shared/`) usado para compartilhar o laudo. Salvar em Downloads não exige permissão (usa `MediaStore` no Android 10+).

Identificação do Shell: `applicationId = com.netfloor.netfloor_shell`, rótulo **NetFloor**, `versionName 3.2.0`, `versionCode 5`. O APK é assinado com a **chave de debug** do Flutter (adequado a uso pessoal; a Play Store exigiria uma keystore própria).

## 7.4 Desenvolvimento local

```
cd "C:\Users\Roger\Desktop\Nova pasta\netfloor"
flutter pub get
run_web.bat      :: flutter run -d web-server --web-port 8090 --web-hostname 0.0.0.0
```

Acesse `http://localhost:8090`. Na primeira compilação a tela pode ficar preta por ~1 minuto; recarregue a página quando o terminal indicar que o app está sendo servido. Com `--web-hostname 0.0.0.0`, outros aparelhos da mesma rede acessam pelo IP do computador (pode ser necessário liberar a porta no Firewall do Windows).

## 7.5 Build web de produção

```
flutter build web --release --base-href "/netfloor/" --pwa-strategy=none
```

| Opção                                 | Motivo                                                                                                                                                                                                                                                        |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>--base-href "/netfloor/"</code> | O site é servido em um subcaminho do GitHub Pages                                                                                                                                                                                                             |
| <code>--pwa-strategy=none</code>      | Desliga o cache de service worker: cada abertura busca a versão mais recente (base do OTA). Como consequência, o app precisa de internet para abrir. A opção está marcada como <b>obsoleta</b> no Flutter 3.47 (funciona, mas será removida em versão futura) |

## 7.6 Pipeline de implantação contínua (OTA) via GitHub Pages

Não há GitHub Actions: o pipeline é manual e reproduzível, sempre nesta ordem.

1. **Commit e push do código** no `main` (assim as imagens ficam disponíveis em `raw.githubusercontent.com` antes do site que as referencia).

2. **Build web** (seção 7.5).

3. **Publicar** o conteúdo de `build/web` no branch `gh-pages` (histórico descartável, sobrescrito):

```
cd netfloor/build/web
rm -rf .git && git init -q && git checkout -q -b gh-pages
git add -A && git commit -q -m "Deploy: NetFloor vX.Y"
git remote add origin https://github.com/Rogerdev5690/netfloor.git
git push -f origin gh-pages
```

4. **Aguardar** o GitHub Pages (`gh api repos/Rogerdev5690/netfloor/pages/builds/latest --jq .status` até `built`) e conferir o site.

5. O aplicativo instalado carrega a nova versão **na próxima abertura**, sem reinstalar o APK.

## 7.7 Build e versionamento do APK Shell

```
cd "C:\Users\Roger\Desktop\Nova pasta\netfloor_shell"
:: pubspec.yaml -> version: 3.2.0+5 (e kShellVersion em lib/main.dart)
flutter build apk --release
gh release create v3.2.0 build/app/outputs/flutter-apk/app-release.apk#NetFloor-v3.2.0-shell.apk ^
--repo Rogerdev5690/netfloor --title "NetFloor Shell v3.2.0" --notes "..."
```

| Release | Conteúdo                                                                                                                                           |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| v1.0.0  | Primeiro APK nativo em Flutter (histórico; obsoleto)                                                                                               |
| v2.0.0  | Primeiro Shell WebView (arquitetura OTA)                                                                                                           |
| v3.0.0  | Shell com diagnóstico nativo (ponte JS, Wi-Fi, permissões, seletor de arquivos)                                                                    |
| v3.1.0  | Seleção múltipla de imagens no seletor de arquivos                                                                                                 |
| v3.2.0  | Entrega de arquivos ( <code>saveFile</code> : Downloads e compartilhamento), seletor de <code>.json</code> para importar projetos ( <b>atual</b> ) |

**Regra de versão do Shell:** incrementar o `versionCode` (o número após `+`) a cada APK novo; mudar o número maior quando houver novos recursos nativos. O APK atualiza o anterior por cima (mesmo `applicationId` e mesma chave de assinatura).

**Instalação no celular:** baixar o APK do release e instalar (o Android pede para permitir "instalar apps desconhecidos"), ou, com Depuração USB ativa, `adb install -r app-release.apk`. Em aparelhos Xiaomi pode ser necessário habilitar também "Instalar via USB".

## 7.8 Checklist de release

- [ ] `flutter analyze` sem erros no app web e no Shell.
- [ ] Teste no navegador: simulador, troca de andares, upload, ferramentas (parede, régua, ponto), troca de banda e de modo de visualização, persistência (recarregar a página), exportar/importar, geração do laudo e as três abas do diagnóstico em simulação.
- [ ] Commit e push no `main`; build web; deploy no `gh-pages`; site verificado.
- [ ] Se houve mudança nativa: novo APK, `versionCode` incrementado, release publicado.
- [ ] **Atualizar** `docs/DOCUMENTACAO.md` e **regenerar o PDF** (seção 9).

## 7.9 Solução de problemas

| Sintoma                                                                         | Causa                                                                          | Solução                                                                                |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| "Building with plugins requires symlink support"                                | Plugins com pasta <code>windows/</code> exigem o Modo Desenvolvedor do Windows | O projeto web não usa a plataforma Windows; a pasta <code>windows/</code> foi removida |
| Gradle: Failed to find target with hash string 'android-37'                     | <code>permission_handler</code> 13 exige <code>compileSdk 37</code>            | Usar <code>permission_handler</code> 12.0.1                                            |
| Tela preta na 1ª abertura do servidor local                                     | Compilação inicial do Flutter Web                                              | Aguardar e recarregar                                                                  |
| Espectro vazio no Android                                                       | Localização negada ou GPS desligado; limite de varreduras                      | Conceder permissão, ligar o GPS e tocar em <b>Escanear</b>                             |
| <code>adb devices</code> não lista o celular                                    | Depuração USB desligada ou modo USB incorreto                                  | Ativar Depuração USB (e "Instalar via USB" em Xiaomi) e autorizar o computador         |
| Imagem da planta não aparece                                                    | URL do <code>raw.githubusercontent.com</code> ainda inexistente                | Fazer push no <code>main</code> ; o app usa o asset embutido como reserva              |
| <code>withData</code> não existe no <code>file_picker</code>                    | API mudou na versão 13                                                         | Usar <code>readAsBytes()</code> do <code>PlatformFile</code>                           |
| Não consigo arrastar roteadores dentro do <code>InteractiveViewer</code>        | O reconhecedor de escala vence o <code>onPanUpdate</code> do filho             | Tratar toques com <code>Listener</code> (ponteiros crus), como no simulador            |
| <code>fontSize != null ...</code> <code>fontSizeFactor</code> ao trocar de tema | <code>TextTheme.apply(fontSizeFactor)</code> não aceita estilos sem tamanho    | Escalar o texto por <code>MediaQuery.textScaler</code>                                 |
| Texto ilegível em cartões claros no tema escuro                                 | Cores de fundo fixas (âmbar, verde, laranja) com texto herdado do tema         | Fixar <code>Colors.black87</code> nesses cartões                                       |
| PDF muito grande (> 5 MB)                                                       | Imagem bruta RGBA incorporada                                                  | Codificar o mapa em JPEG antes de incorporar                                           |
| "Atualize o app NetFloor Shell" ao gerar o laudo no celular                     | Shell anterior à 3.2.0 não tem <code>saveFile</code>                           | Instalar o APK 3.2.0                                                                   |

## 7.10 Segurança e privacidade

- Nenhum dado do usuário é enviado a servidores: projetos, plantas enviadas, laudos, logos e assinaturas ficam no IndexedDB do aparelho (ou no arquivo que o usuário exportar); leituras Wi-Fi são exibidas localmente. Limpar os dados do site/aplicativo apaga os projetos salvos: exporte o `.json` para ter cópia.
- O Shell só carrega o domínio do projeto; a ponte nativa não é acessível a outros sites.
- O ping usa apenas os destinos exibidos (gateway local e 8.8.8.8).
- As permissões de localização são usadas exclusivamente porque o Android as exige para varredura Wi-Fi.

## 8. Limitações Conhecidas e Pendências

| Tema                                 | Descrição                                                                                                                                                                             |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Plantas provisórias                  | <code>planta_01..03</code> e <code>edificio_corporativo</code> aguardam imagens limpas; usam desenho vetorial                                                                         |
| Proporção fixa por planta            | O <code>aspectRatio</code> de cada planta é definido no código; ao trocar um PNG por outro de proporção diferente, o valor deve ser ajustado                                          |
| Paredes aproximadas                  | As da biblioteca são visuais; as das plantas enviadas dependem do usuário desenhá-las                                                                                                 |
| Modelo de RF simplificado            | Ver seção 3.9; o nível simulado não é o RSSI real                                                                                                                                     |
| Uso offline                          | Os <b>dados</b> ficam salvos offline, mas com <code>--pwa-strategy=none</code> o app ainda precisa de internet para <b>abrir</b> (o service worker foi desligado para garantir o OTA) |
| Assinatura eletrônica simples        | É a imagem do traço, sem certificado digital                                                                                                                                          |
| <code>saveFile</code> no Shell 3.2.0 | Compilado; ainda não validado em aparelho                                                                                                                                             |
| Medições no navegador                | Simuladas (marcadas como tal no laudo); medição real exige o Shell                                                                                                                    |
| Assinatura do APK                    | Chave de debug: não serve para Play Store                                                                                                                                             |
| Testes automatizados                 | Inexistentes; validação é manual                                                                                                                                                      |
| Histórico do Git                     | Arquivos removidos (por exemplo, com marca d'água) continuam nos commits antigos                                                                                                      |
| Pasta <code>netfloor/android/</code> | Legado do APK nativo v1.0.0; o projeto atual depende de <code>dart:js_interop</code> e só compila para Web                                                                            |
| Seleção múltipla no Shell 3.1.0      | Compilada, ainda não validada em aparelho                                                                                                                                             |

## 9. Manutenção desta Documentação (Regra de Projeto)

---

Esta é a **versão base** da documentação. A cada nova atualização do aplicativo:

1. Implementar a alteração pedida.
2. Atualizar a(s) seção(ões) correspondente(s) de `docs/DOCUMENTACAO.md`, incluindo o **histórico de alterações** (seção 6.7) e, se necessário, a tabela de releases (7.7).
3. Ajustar as três marcas no topo do arquivo (`doc-version`, `doc-revision`, `doc-date`).
4. Regenerar o PDF e entregá-lo ao usuário:

```
python "C:\Users\Roger\Desktop\Nova pasta\netfloor\docs\build_pdf.py"
```

O script converte o Markdown em HTML (com fórmulas em MathML), imprime em PDF pelo Chrome em modo headless, calcula os números de página do sumário, acrescenta cabeçalho/rodapé com numeração e cria os marcadores (bookmarks) do PDF. Dependências Python: `markdown`, `latex2mathml`, `pygments`, `pypdf`, `reportlab`.

## Apêndice A — Glossário

| Termo                     | Significado                                                                                                                         |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <b>dBm</b>                | Decibel-miliwatt: unidade logarítmica de potência; valores mais próximos de 0 são sinais mais fortes (–50 dBm é melhor que –80 dBm) |
| <b>RSSI</b>               | Received Signal Strength Indicator: nível de sinal recebido pelo aparelho                                                           |
| <b>SSID / BSSID</b>       | Nome da rede / identificador (MAC) do ponto de acesso específico                                                                    |
| <b>PHY (velocidade)</b>   | Taxa de dados negociada na camada física entre aparelho e roteador (não é a velocidade da Internet)                                 |
| <b>Canal / largura</b>    | Faixa de frequência usada por uma rede (20, 40, 80 ou 160 MHz)                                                                      |
| <b>Ray-casting</b>        | Traçado de um raio entre dois pontos para contar obstáculos cruzados                                                                |
| <b>2.5D</b>               | Simulação em planos empilhados (andares) sem modelo 3D completo                                                                     |
| <b>Laje</b>               | Estrutura horizontal de concreto entre pavimentos                                                                                   |
| <b>PWA</b>                | Progressive Web App: aplicativo web instalável                                                                                      |
| <b>OTA</b>                | Over-the-air: atualização sem reinstalar o aplicativo                                                                               |
| <b>WebView</b>            | Componente do Android que exibe páginas web dentro de um app                                                                        |
| <b>Mesh</b>               | Rede com vários nós que cooperam para ampliar a cobertura                                                                           |
| <b>Calibração (régua)</b> | Ajuste da escala metros/pixel a partir de uma distância real conhecida                                                              |
| <b>IndexedDB</b>          | Banco de dados do navegador/WebView que guarda os projetos offline                                                                  |
| <b>Laudo de vistoria</b>  | Documento PDF entregue ao cliente com mapas, medições e assinatura                                                                  |
| <b>Gateway</b>            | Roteador local da rede (destino do ping "local")                                                                                    |

## Apêndice B — Referência rápida de constantes

| Constante                                        | Valor                                                          | Onde                                                                                               |
|--------------------------------------------------|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>FloorPlanDef.defaultWidthM</code>          | 8 m (padrão)                                                   | Largura estimada da planta até calibrar (12/12/10/20 nas plantas 01/02/03/edifício; 10 em uploads) |
| <code>kFloorHeightM</code>                       | 3,0 m                                                          | Distância vertical entre andares                                                                   |
| <code>kSlabLossDb</code>                         | 15 dB                                                          | Perda por laje (todas as bandas)                                                                   |
| <code>kStrongDbm</code> / <code>kWeakDbm</code>  | 12 / -10 dBm                                                   | Limiares Forte / Ruim                                                                              |
| <code>kRfBands</code>                            | 2.4: (0, 22, 1,00) · 5: (-6,4, 24, 1,35) · 6: (-7,9, 25, 1,50) | ( $\Delta$ ref dB, k, fator de paredes)                                                            |
| Materiais de parede                              | Gesso 2 · Cerâmico 4 · Vidro 5 · Concreto 12–15 (13,5) dB      | <code>kWallMaterials</code>                                                                        |
| Zoom máximo                                      | 12×                                                            | <code>InteractiveViewer.maxScale</code>                                                            |
| Autosave                                         | 0,8 s                                                          | Após a última alteração                                                                            |
| Largura do mapa no PDF                           | 1000 px, JPEG q88                                              | <code>renderFloorRaster</code>                                                                     |
| Pedaço de arquivo na ponte                       | 256 KB (base64)                                                | <code>FileIO.deliver</code> → <code>saveFile</code>                                                |
| <code>_kDbmFloor</code> / <code>_kDbmCeil</code> | -30 / +26 dBm                                                  | Escala de cor do mapa de calor                                                                     |
| <code>kDefaultHeatOpacity</code>                 | 0,48                                                           | Opacidade padrão do calor                                                                          |
| <code>cell</code> (grade)                        | 8 px (proporcional no PDF)                                     | Amostragem do mapa de calor                                                                        |
| <code>kMaxSamples</code>                         | 60                                                             | Amostras exibidas nos gráficos de linha                                                            |
| <code>kDnsHost</code>                            | 8.8.8.8                                                        | Alvo do ping de Internet                                                                           |
| Intervalo de varredura                           | 30 s                                                           | Aba Espectro                                                                                       |
| Intervalo de sinal e ping                        | 1 s                                                            | Abas Sinal e Latência                                                                              |